University of Cape Town ECO5040W
Project Resources

1. Understanding XRP, Trust Lines & IOUs

Before writing any code, make sure you and your team actually understand *why* XRPL works the way it does.

Core model

- On the XRP Ledger, XRP is the only asset every account can hold natively. Everything else on the chain: RLUSD, USD, EUR, in-game currencies, whatever, is an **issued currency**, sometimes called a **trust line token** or, informally, an **IOU**. An issued token is literally a promise from an issuing account that it owes the holder that amount of the underlying asset.
- Because issued tokens are IOUs, you cannot receive one until you explicitly opt in. That opt-in is a **trust line**: an accounting relationship between your account and the issuer's account, created with a `TrustSet` transaction, that says "I'm willing to hold up to X of this issuer's token."
- Creating a trust line reserves **2 XRP** in your account (locked, not spent, you get it back if you remove the trust line). Make sure any test wallet you fund has enough balance for this on top of the base account reserve.
- **Rippling** is a setting on an XRPL account that lets tokens with the same currency code move between different holders of the same issuer without passing back through the issuer explicitly. An issuer normally enables "Default Ripple" so its customers can transact with each other; ordinary holders should generally leave **rippling off** on their own accounts. You don't need to implement this yourselves, but you should understand it well enough to explain it in your spec.

Some Resources:

1. Fungible Tokens trust line tokens vs. MPTs, when to use which.
2. Trust Lines
3. Rippling
4. Stablecoins on XRPL and Stablecoin Settings how an issuer configures itself (Default Ripple, freeze, clawback) and what that means for you as a *holder* of RLUSD rather than an issuer.
5. Authorized Trust Lines
6. Set Up Your First XRPL Trustline. A good third-party explainer if you want it in plainer language.

---

2. Getting an XRPL Wallet: Graphically (Xaman)

Xaman (formerly Xumm) is the standard mobile/desktop wallet for XRPL, and it's the fastest way to get started without a single line of code.

1. Install Xaman from the App Store / Google Play
2. On first launch, choose to create a new wallet. **Write down and securely store the recovery/secret words**
3. Switch Xaman into developer/Testnet mode: **Settings → Advanced → scroll down → enable "Developer Mode."** (full walkthrough)
4. Back on the home screen, tap the **network switcher (top right)** and select **Testnet**.
5. Fund the account from a Testnet faucet, the app will guide you to do this or you can get from the public faucet
6. To set up a trust line to the RLUSD issuer visit the issuer faucet and follow the instructions, connecting your **Xaman** wallet and receiving your 10 RLUSD.

This graphical flow is exactly what your **users** will effectively be doing under the hood when your backend creates/manages their custodial wallets. This is just a nice way to get started and get a feel for what you are actually going to be implementing.

---

3. Getting an XRPL Wallet: Programmatically

For your actual platform, wallet creation and trust lines need to happen in code.

SDKs

- Python: xrpl-py
- JavaScript/TypeScript: `xrpl.js`, if any part of your team is more comfortable in Node for the settlement worker.

Programmatic flow (Python, `xrpl-py`):

1. Generate a wallet and fund it via the Testnet faucet helper (`generate_faucet_wallet`)
2. Submit a `TrustSet` transaction from the new wallet to the RLUSD issuer address, with a currency code and a limit amount. (This is a trust line!)
3. Submit a `Payment` transaction (from the issuer, or from a funded holder, depending on the flow you're testing) to move RLUSD.
4. Store the resulting transaction hash and validate it against the ledger.

Some Resources:

- Get Started Using Python - XRPL
- Create Trust Line and Send Currency Using Python walks you through exactly the `TrustSet` + `Payment` sequence you need.
- Hands-On: Issuing and Transferring a Stablecoin on XRPL good end-to-end walkthrough
- RippleDevRel/xrpl-js-python-simple-scripts incredible resource, thank me later.

Testnet network details:

- JSON-RPC: `https://s.altnet.rippletest.net:51234/`
- WebSocket: `wss://s.altnet.rippletest.net:51233/`
- Faucets: xrpl.org faucets

---

4. RLUSD vs our own IOU

Since the official RLUSD faucet only allows each wallet to receive about 10$ worth of liquidity per 24 hours, you and your team might run into liquidity issues during testing, we may need to use an alternative IOU. **We have reached out to Ripple to fund an official UCT XRPL wallet.** Ripple will likely fund it, which means we can provide you with RLUSD on the testnet as needed.

Reach out to Marc via LVNMAR013@myuct.ac.za OR contact Marc on WhatsApp if need be for RLUSD liquidity.

However, in the case that Ripple does not fund our wallet in time, we will create an **IOU** token that functions exactly like **RLUSD** except it will be created by us.

What this means for you right now: build your wallet and trust-line code so the **issuer address is a config value, not a hardcoded constant (Remember .env files?)**. Whichever way this goes, switching between the official issuer and a fallback IOU should be a one-line config change, not a rewrite. This is also good practice for the "assumptions and limitations" section of your documentation regardless of which way it goes.

---

5. Programming & Development Resources

Backend / API

- Flask, Django, FastAPI

- Interactive API docs: FastAPI ships OpenAPI/Swagger out of the box

Database

- SQLite is fine for development; consider Postgres if you want something closer to production for the performance-testing component but remember KISS (Keep it simple, stupid!)

Message queue

- RabbitMQ or Redis Streams are the lowest-friction options to stand up locally or on a small cloud instance; Kafka is heavier than you need for this project's scale.

XRPL tooling

- `xrpl-py` / `xrpl.js` — official client libraries.
- XRPL Explorer (switch to Testnet) and Bithomp's Testnet explorer: for eyeballing transactions and trust lines while debugging.
- Xrpl.services a Testnet faucet + a browser-based transaction sandbox, handy for testing a `TrustSet` or `Payment` payload

Deployment

- Render or Railway for hosting your backend(s), both mentioned in the brief and both have reasonable free tiers for a class project

Security (private keys)

- Look at Python's `cryptography` package (Fernet or AES-GCM) for encrypting stored XRPL secrets, and keep the encryption key in a separate store (environment variable / secrets manager) from the database holding the encrypted keys.

Questions on any of the above: raise them in the check-ins , or email Julian (julian.kanjere@uct.ac.za), cc Marc (LVNMAR013@myuct.ac.za).